HashMap Internals

11 min read Updated 28 Jul 2026 Topic 18 of 65

Bookmark Mark done

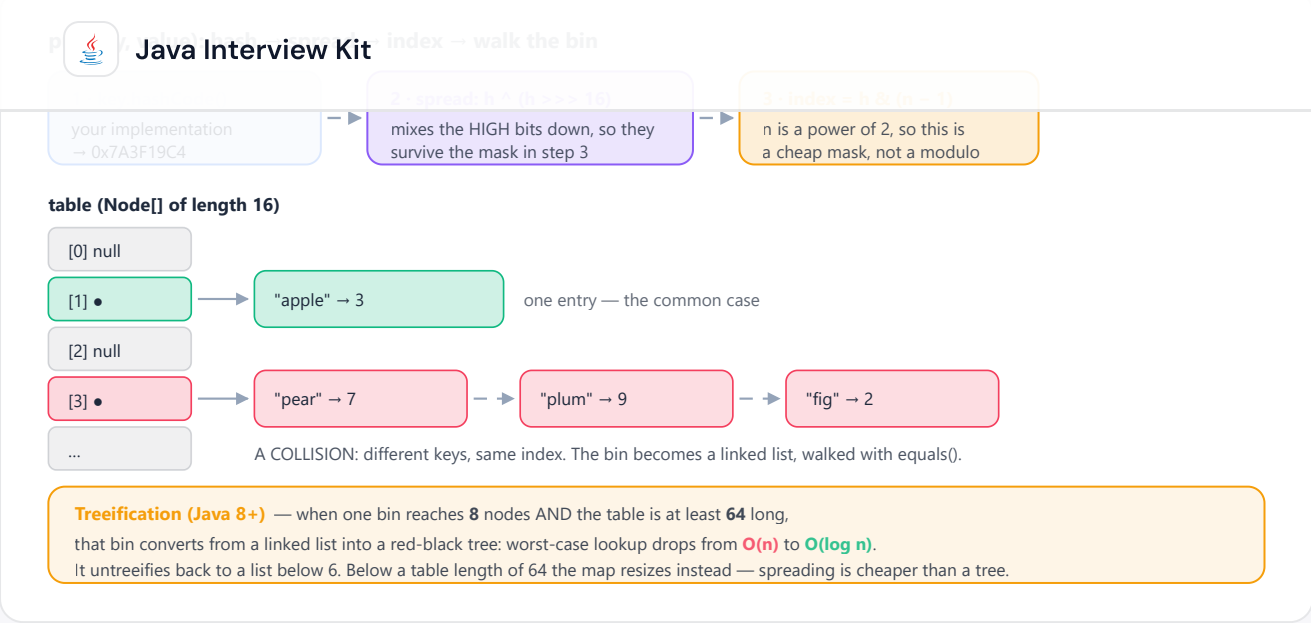
Why it matters

If there is one topic that decides Java interviews, this is it. It is asked at every level, from "how does a `HashMap` work?" for a fresher to "walk me through resize under concurrent writes" for a senior. It rewards genuine understanding and punishes memorisation, because the follow-ups go one layer deeper each time. It is also the payoff for Chapter 2's `equals` / `hashCode` contract — everything here explains *why* that contract exists.

The concept, simply

A `HashMap` is an array of *bins* (the `table`). To store a key it computes a number from the key, turns that into an array index, and puts the entry there. To look one up it repeats the calculation and goes straight to that index — which is why average lookup is O(1) rather than a scan.

When two different keys land on the same index — a **collision**, which is inevitable — the entries are chained in that bin and compared with `equals`.



The full put path. Note step 2: the spread function XORs the high 16 bits down, because step 3's mask would otherwise discard them entirely.

How it works

Step 1 — `hashCode()`**.** Your implementation, returning a 32-bit int.

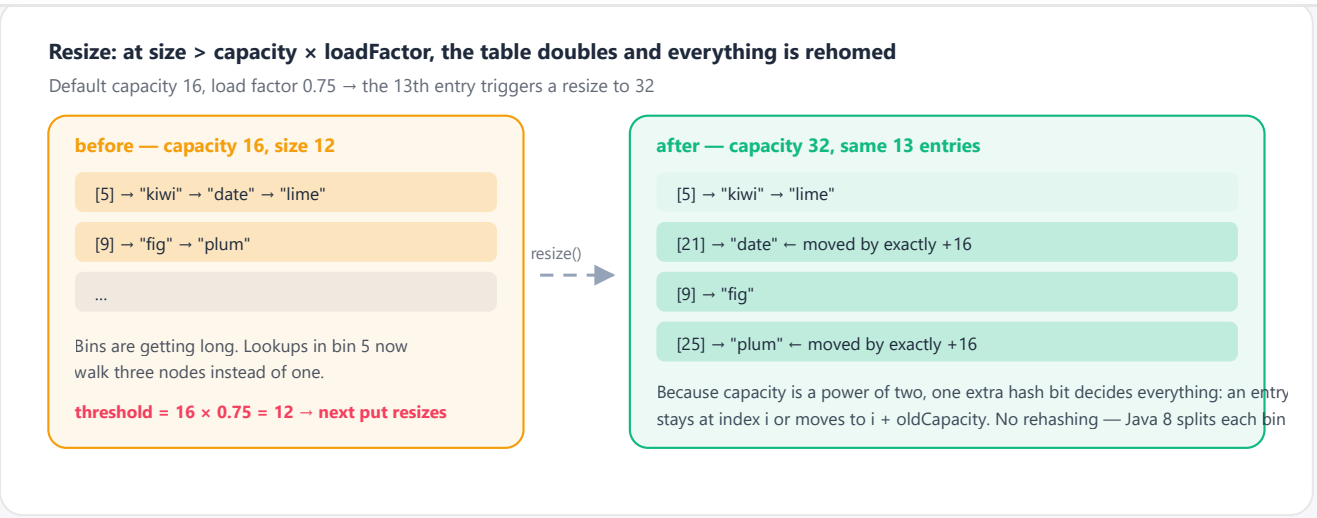
Step 2 — spreading. `HashMap` does not use your hash directly. It applies `h ^ (h >>> 16)`, XOR-ing the top 16 bits into the bottom 16. The reason is step 3: because the table length is a power of two, the index is computed with a bitmask that keeps only the *low* bits. Without spreading, two keys differing only in their high bits would always collide. This one line rescues mediocre `hashCode` implementations.

Step 3 — index. `index = hash & (table.length - 1)`. Because the length is always a power of two, subtracting one gives a mask of low bits, and a bitwise AND is far cheaper than a modulo. That is the whole reason capacities are powers of two.

Step 4 — walk the bin. If the bin is empty, store the node. Otherwise compare each existing entry: first by hash (a cheap int comparison), then by `equals`. A match replaces the value; no match appends a new node.

Treeification (Java 8). Before Java 8, a heavily-collided bin was a linked list and lookup degraded to O(n) — which was a real denial-of-service vector, since an attacker who could choose keys could force every entry into one bin. Java 8 added a fix: when a bin reaches **8** nodes *and* the table is at least **64** long, the bin converts into a red-black tree, giving O(log n) worst case. It reverts to a list below 6 nodes. Below a table length of 64, the map resizes instead — spreading entries is cheaper than building a tree.

Resize: When `size > capacity × loadFactor` (default 0.75), the table doubles:



Doubling means one extra hash bit decides each entry's fate: it stays at index i, or moves to i + oldCapacity. Java 8 splits each bin into a "low" and "high" list in a single pass, with no rehashing.

Code examples

Sizing, which is the practical takeaway:

JAVA Copy

```java
// Default: capacity 16, load factor 0.75 → resizes at 13 entries.
Map<String, Integer> m = new HashMap<>();

// Storing ~1000 entries with the default means resizing at 13, 25, 49, 97
// 193, 385, 769 — seven table allocations and seven full rehomings.
// Presize instead. Divide by the load factor so it never resizes:
Map<String, Integer> sized = new HashMap<>((int) (1000 / 0.75f) + 1);

// Java 19+ makes this readable and removes the arithmetic entirely:
Map<String, Integer> better = HashMap.newHashMap(1000);
```

The modern API most people still do not use:

JAVA Copy

```java
Map<String, List<String>> index = new HashMap<>();

// ✗ The old dance — two lookups and a null check.
```

```
// ✓ One call, one lookup.
index.computeIfAbsent(key, k -> new ArrayList<>()).add(value);

// Counting:
Map<String, Integer> counts = new HashMap<>();
counts.merge(word, 1, Integer::sum);            // ✓ the idiomatic word cc
counts.put(word, counts.getOrDefault(word, 0) + 1);   // equivalent, two l

// Careful: computeIfAbsent removes the entry if the function returns null
// and it is illegal to modify the same map inside the function — that thr
// ConcurrentModificationException in Java 9+.
```

What a bad `hashCode` does to you:

JAVA Copy

```
class BadKey {
    final String id;
    BadKey(String id) { this.id = id; }
    @Override public int hashCode() { return 1; }        // ✗ legal, catas
    @Override public boolean equals(Object o) {
        return o instanceof BadKey k && id.equals(k.id);
    }
}
// Every key lands in bin 1. The map degrades to a single chain — and, pas
// entries with a table of 64+, to a single red-black tree. Lookup becomes
// O(log n) at best instead of O(1). The contract is satisfied; the
// performance is destroyed. Correct is not the same as good.
```

Under the hood

Why 0.75? It is the documented trade-off point between space and time. Lower means more empty buckets and fewer collisions; higher means a denser table and longer chains. The javadoc notes that with a good hash function, entry counts per bin follow a Poisson distribution at this load factor, and the probability of a bin reaching 8 entries is

or pathological hashes, not a normal occurrence.

The Java 7 infinite loop is the best-known `HashMap` war story, and worth being able to tell. In Java 7, resize re-inserted entries at the head of each new bin, which reversed each chain. Two threads resizing simultaneously could weave a *cycle* into a bin's linked list, and a later `get()` would spin forever at 100% CPU — a hung production server with no exception in any log. Java 8's split-in-place resize preserves relative order and does not reverse chains, so that specific failure is gone. But `HashMap` is still **not thread-safe**: concurrent writes can still lose updates and corrupt the size. Use `ConcurrentHashMap`.

Hash-collision DoS was the security motivation for treeification. Before Java 8, an attacker who could control map keys — form parameters, JSON fields, HTTP headers — could submit thousands of keys engineered to collide, turning every insertion into a linear scan and pinning a CPU with a small request. Treeification caps that at O(log n). It is a good answer to "why did they add red-black trees?", because the honest answer is security rather than everyday performance.

Common mistakes

- **Mutable keys.** Change a field `hashCode` uses and the entry is stranded in the wrong bin — unreachable even by the same key object.
- **Overriding `equals` without `hashCode`.** Chapter 2's contract; here you can see the exact mechanism it breaks.
- **Using `HashMap` from multiple threads.** Not thread-safe. Lost updates, corrupt size, and historically an infinite loop.
- **Not presizing.** Loading 10,000 entries into a default map costs ten resizes and ten full rehomings.
- **A constant or near-constant `hashCode`.** Legal, and it destroys the data structure.
- **Relying on iteration order.** It is bucket order and can change on resize. Use `LinkedHashMap` if order matters.
- **Modifying the map inside `computeIfAbsent`.** Throws `ConcurrentModificationException` in Java 9+.

Q: "How does `HashMap` work internally?"

A: It is an array of bins. On `put`, it calls `hashCode()`, applies a spread function (`h ^ h >>> 16`) to mix the high bits down, then masks with `length - 1` to get the index. If that bin is empty it stores a node; otherwise it walks the bin comparing hash first and then `equals`, replacing on a match and appending otherwise. `get` repeats the same calculation. Average O(1); worst case O(n) before Java 8, O(log n) after treeification.

Q: "What is the spread function for?"

A: The index only uses the low bits, because the mask is `length - 1` on a power-of-two table. So two hash codes that differ only in their high bits would always collide. XOR-ing the top 16 bits down means the high bits still influence the index. It is a cheap way to tolerate mediocre `hashCode` implementations.

Q: "Why is capacity always a power of two?"

A: So the index can be computed with `hash & (length - 1)` instead of a modulo, which is much cheaper. It also makes resize elegant: doubling means exactly one more hash bit participates, so each entry either stays at index *i* or moves to *i* + oldCapacity — the bin can be split in one pass with no rehashing.

Q: "What changed in Java 8?"

A: Treeification and a safer resize. A bin with 8 or more nodes, in a table of at least 64, converts from a linked list to a red-black tree, so worst-case lookup drops from O(n) to O(log n) — motivated by hash-collision denial-of-service attacks. And resize now splits each bin in place preserving order, instead of re-inserting at the head, which eliminated the Java 7 concurrent-resize infinite loop.

A: When size exceeds capacity × load factor — 16 × 0.75 = 12 by default, so the 13th entry triggers it. The table doubles and every entry is rehomed. That is O(n) work, and repeatedly for a growing map, so I presize with `HashMap.newHashMap(n)` (Java 19+) or `new HashMap<>((int)(n / 0.75f) + 1)` when the size is known.

Q: "What happens with a bad `hashCode`?"

A: Correctness survives; performance collapses. If every key returns the same value, all entries land in one bin and lookup becomes linear — or logarithmic after treeification. The map still returns the right answers, which is what makes it so insidious: the only symptom is that an O(1) structure is behaving like a list.

Q: "Why is `HashMap` not thread-safe, and what would you use?"

A: No synchronisation at all, so concurrent writes can lose entries, corrupt the size counter, and in Java 7 could weave a cycle into a bin during simultaneous resizes and hang a thread at 100% CPU forever. Java 8's resize fixed that specific loop but the class is still unsafe. Use `ConcurrentHashMap` — bin-level locking, lock-free reads, and it scales with cores.

Cheat-card

- Path: `hashCode()` → **spread** `h ^ (h>>>16)` → **index** `h & (n-1)` → walk bin (hash, then `equals`).
- Capacity is a **power of two** → mask instead of modulo, and a clean resize split.
- Defaults: **capacity 16, load factor 0.75** → resize at 13.
- Resize doubles; an entry stays at *i* or moves to *i* + oldCapacity. No rehashing.
- **Treeify at 8 nodes if table ≥ 64**; untreeify at 6. O(n) → O(log n). Motivated by collision DoS.
- Presize: `HashMap.newHashMap(n)` (19+) or `(int)(n/0.75f)+1`.
- Use `computeIfAbsent` / `merge` / `getOrDefault`, not get-check-put.

QUICK CHECK — no marks, just for you.

1. Why does HashMap apply h ^ (h >>> 16) to the key's hash code?

- ◯ Because the bucket index is computed by masking the LOW bits, so mixing the high bits down means they still influence which bucket is chosen
- ◯ To make the hash code positive
- ◯ To compress the hash into fewer bits
- ◯ To detect collisions

Check answer

2. When does a HashMap bin convert from a linked list into a red-black tree?

- ◯ Never — that is a ConcurrentHashMap feature
- ◯ When the bin reaches 8 nodes AND the table is at least 64 entries long
- ◯ As soon as any collision occurs
- ◯ When the map exceeds its load factor

Check answer

3. With the default capacity of 16 and load factor 0.75, which insertion triggers the first resize?

- ◯ The 8th

- The 13th — the threshold is 16 x 0.75 = 12
- The 12th

Check answer

Previous
ArrayList vs LinkedList

Next
LinkedHashMap and TreeMap

LEARN MORE

ARTICLES & DOCS

- HashMap — API docs
  docs.oracle.com
- Object.hashCode contract — API docs
  docs.oracle.com

VIDEOS



How HashMap Internally Works in Java With Animation | Popular Java Interview QA | Java Techie
Java Techie



How HashMap in Java works internally? - Explained with animation.
Live To Code

Hand-picked references for this lesson.